Multitask-Based Evaluation of Open-Source LLM on Software Vulnerability

Xin Yin, Chao Ni, and Shaohua Wang

***Abstract*—This paper proposes a pipeline for quantitatively evaluating interactive Large Language Models (LLMs) using publicly available datasets. We carry out an extensive technical evaluation of LLMs using Big-Vul covering four different common software vulnerability tasks. This evaluation assesses the multi-tasking capabilities of LLMs based on this dataset. We find that the existing state-of-the-art approaches and pre-trained Language Models (LMs) are generally superior to LLMs in software vulnerability detection. However, in software vulnerability assessment and location, certain LLMs (e.g., CodeLlama and WizardCoder) have demonstrated superior performance compared to pre-trained LMs, and providing more contextual information can enhance the vulnerability assessment capabilities of LLMs. Moreover, LLMs exhibit strong vulnerability description capabilities, but their tendency to produce excessive output significantly weakens their performance compared to pre-trained LMs. Overall, though LLMs perform well in some aspects, they still need improvement in understanding the subtle differences in code vulnerabilities and the ability to describe vulnerabilities to fully realize their potential. Our evaluation pipeline provides valuable insights into the capabilities of LLMs in handling software vulnerabilities.**

***Index Terms*—Software vulnerability analysis, large language model.**

I. Introduction

SOFTWARE Vulnerabilities (SVs) can expose software systems to risk situations and eventually cause huge economic losses or even threaten people's lives. Therefore, completing software vulnerabilities is an important task for software quality assurance (SQA). Generally, there are many important software quality activities for software vulnerabilities such as SV detection, SV assessment, SV location, and SV description. The relationship among the SQA activities is intricate and interdependent and can be illustrated in Fig. 1. SV detection serves as the initial phase, employing various tools and techniques to identify potential vulnerabilities within the software. Once detected, the focus shifts to SV assessment, where the severity and potential impact of each vulnerability are meticulously evaluated. This critical evaluation informs the subsequent steps in the process. SV location follows the assessment, pinpointing the exact areas within the software's code or architecture where vulnerabilities exist. This step is crucial for precise remediation efforts and to prevent the recurrence of similar vulnerabilities in the future. The intricacies of SV location feed into the comprehensive SV description, which encapsulates detailed information about each vulnerability, including its origin, characteristics, and potential exploits. In essence, the synergy among SV detection, SV assessment, SV location, and SV description creates a robust pipeline for addressing software vulnerabilities comprehensively. This systematic approach not only enhances the overall quality of the software but also fortifies it against potential threats, thereby safeguarding against economic losses and potential harm to individuals. As a cornerstone of software quality assurance, the seamless integration of these activities underscores the importance of a proactive and thorough approach to managing software vulnerabilities in today's dynamic and interconnected digital landscape.

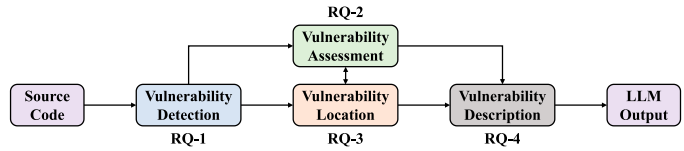


Fig. 1. The relationship among software vulnerability analysis activities.

Recently, Large Language Models (LLMs) [1] have been widely adopted since the advances in Natural Language Processing (NLP) which enable LLM to be well-trained with both billions of parameters and billions of training samples, consequently bringing a large performance improvement on tasks adopted by LLMs. LLMs can be easily used for a downstream task by being fine-tuned [2] or being prompted [3] since they are trained to be general and they can capture different knowledge from various domain data. Fine-tuning is used to update model parameters for a particular downstream task by iterating the model on a specific dataset while prompting can be directly used

Received 22 January 2024; revised 20 August 2024; accepted 13 September 2024. Date of publication 7 October 2024; date of current version 14 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62202419, in part by the Fundamental Research Funds for the Central Universities under Grant 226-2022-00064, in part by Zhejiang Provincial Natural Science Foundation of China under Grant LY24F020008, in part by Ningbo Natural Science Foundation under Grant 2022J184, and in part by the State Street Zhejiang University Technology Center. Recommended for acceptance by C. Treude. *(Corresponding author: Chao Ni.)*

Xin Yin is with the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou 310007, China (e-mail: xyin@zju.edu.cn).

Chao Ni is with the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou 310007, China, and also with Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou 310051, China (e-mail: chaoni@zju.edu.cn).

Shaohua Wang is with the Central University of Finance and Economics, Beijing 100081, China (e-mail: davidshwang@ieee.org).

Digital Object Identifier 10.1109/TSE.2024.3470333

0098-5589 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

by providing natural language descriptions or a few examples of the downstream task. Compared to prompting, fine-tuning is expensive since it requires additional model training and has limited usage scenarios, especially in cases where sufficient training datasets are unavailable.

LLMs have demonstrated remarkable language comprehension and generation capabilities, and have been able to perform well on a variety of natural language processing tasks, such as text summarization [4]. Given the outstanding performance of LLMs, there is a growing focus on exploring their potential in software engineering tasks and seeking new opportunities to address them. Currently, as more and more LLMs designed for software engineering tasks are deployed [5], [6], [7], [8], [9], [10], [11], many research works focused on the application of LLMs in the software engineering domain [12], [13], [14], [15], [16]. However, in the existing literature, adequate systematic reviews and surveys have been conducted on LLMs in areas such as generating high-quality code and high-coverage test cases [17], [18], but a systematic review and evaluation of open-source LLMs in the field of software vulnerability is still missing.

In this paper, we focus on evaluating LLMs' performance in various software vulnerability (SV)-related tasks in few-shot and fine-tuning settings to obtain a basic, comprehensive, and better understanding of their multi-task ability, and we aim to answer the following research questions.

- **RQ-1: How do LLMs perform on vulnerability detection?** Software Vulnerabilities (SVs) can expose software systems to risk situations and consequently software function failure. Therefore, detecting these SVs is an important task for software quality assurance. We aim to explore the ability of LLMs on vulnerability detection as well as the performance difference compared with state-of-the-art approaches and pre-trained Language Models (LMs).
- **RQ-2: How do LLMs perform on vulnerability assessment?** In practice, due to the limitation of SQA resources [19], it is impossible to treat all detected SVs equally and fix all SVs simultaneously. Thus, it is necessary to prioritize these detected software vulnerabilities for better treatment. An effective solution to prioritize those SVs is to use one of the most widely known SV assessment frameworks CVSS (Common Vulnerability Scoring System) [20], which characterizes SVs by considering three metric groups: Base, Temporal, and Environmental. The metrics that are in the groups can be further used as the criterion for selecting serious SVs to fix early. Therefore, we aim to explore the ability of LLMs to assess vulnerabilities and compare their performance with pre-trained LMs.
- **RQ-3: How do LLMs perform on vulnerability location?** Identifying the precise location of vulnerabilities in software systems is of critical importance for mitigating risks and improving software quality. The vulnerability location task involves pinpointing these weaknesses accurately and helps to narrow the scope for developers to fix problems. Therefore, we aim to investigate LLMs' capability in effectively identifying the precise location of vulnerabilities in software systems, alongside evaluating their performance against state-of-the-art approaches and pre-trained LMs.
- **RQ-4: How do LLMs perform on vulnerability description?** The vulnerability description task focuses on conveying a detailed explanation of these identified issues in the source codes and helps participants to better understand the risk as well as its impacts. Understanding the intricacies of vulnerabilities in software systems plays a pivotal role in alleviating risks and bolstering software quality. The vulnerability description task focuses on conveying a detailed explanation of these identified issues in the source codes and helps participants to better understand the risk as well as its impacts. Our goal is to evaluate LLMs' ability to effectively generate vulnerability descriptions within software systems and compare their performance with that of pre-trained LMs.

To extensively and comprehensively analyze the LLMs' ability, we use a large-scale dataset containing real-world project vulnerabilities (named Big-Vul [21]). We carefully design experiments to discover the findings by answering four RQs. The main contribution of our work is summarized as follows and takeaway findings are shown in Table I. Eventually, we present the comparison of LLMs across four software vulnerability tasks under different settings, as well as the impact of varying model sizes on performance, as depicted in Fig. 2(a) and 2(b). In summary, the key contributions of this paper include:

- We extensively evaluate the performance of LLMs on different software vulnerability tasks and conduct an extensive comparison among LLMs and learning-based approaches to software vulnerability.
- We design four RQs to comprehensively understand LLMs from different dimensions, and provide detailed results with examples.
- We release our replication package for further study [22].

II. Background and Related Work

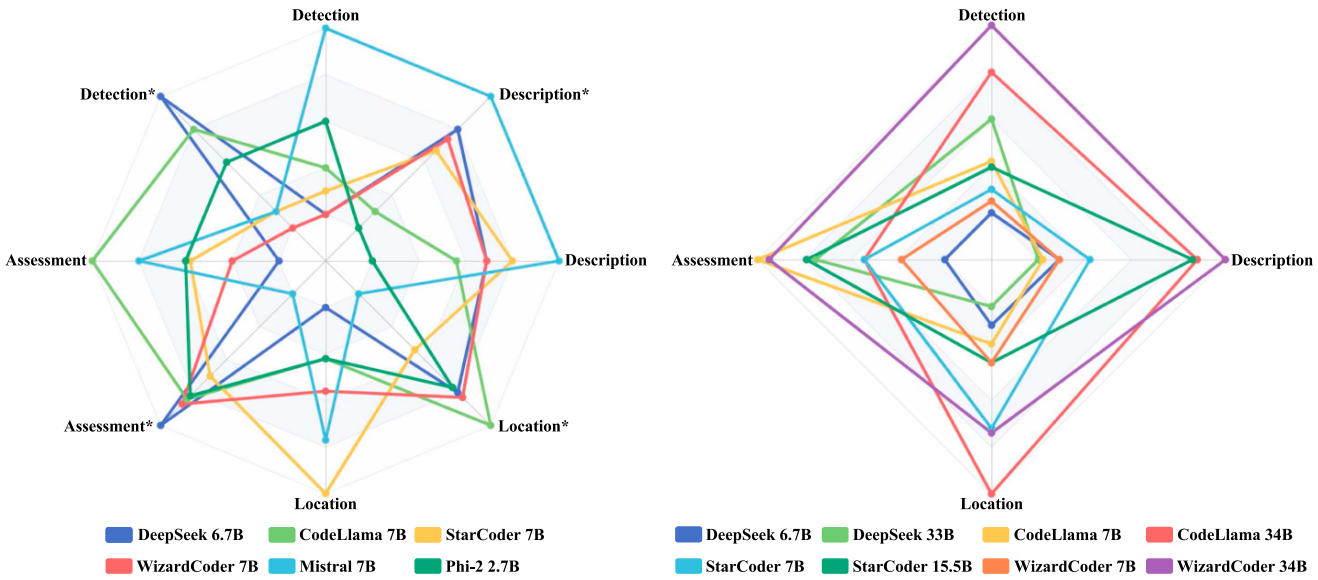
A. Large Language Model

Since the advancements in Natural Language Processing, Large Language Models (LLMs) [1] have seen widespread adoption due to their capacity to be effectively trained with billions of parameters and training samples, resulting in significant performance enhancements. LLMs can readily be applied to downstream tasks through either fine-tuning [2] or prompting [3]. Their versatility stems from being trained to possess a broad understanding, enabling them to capture diverse knowledge across various domains. Fine-tuning involves updating the model parameters specifically for a given downstream task through iterative training on a specific dataset. In contrast, prompting allows for direct utilization by providing natural language descriptions or a few examples of the downstream task. Compared to prompting, fine-tuning is resource-intensive as it necessitates additional model training and is applicable in limited scenarios, particularly when adequate training datasets are unavailable.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TAKEAWAYS: EVALUATING LLMS ON SOFTWARE VULNERABILITY

Dimension	Finding or Guidance
Vulnerability Detection	(1). LLMs can detect vulnerabilities, but fine-tuned LLMs perform weaker than transformer-based approaches. Considering the computational resources and time costs of deploying LLMs, transformer-based approaches for vulnerability detection are a more efficient choice. (2). After fine-tuning, the detection capability of LLMs has improved. Larger models usually perform better, but performance can also be influenced by model design and pre-training data. Therefore, fine-tuning the LLM on domain-specific data before using it as a vulnerability detector is necessary. (3). In general, different LLMs complementing each other, while CodeLlama obtains better performance in terms of F1-score, Precision, and Recall.
Vulnerability Assessment	(4). Overall, fine-tuned code-related LLMs outperform pre-trained language models in vulnerability assessment. When resources permit, fine-tuning DeepSeek-Coder 6.7B for vulnerability assessment is optimal, as it outperforms the pre-trained language models across four metrics. (5). LLMs have the capacity for assessment of vulnerability severity based on source code, and can be improved by providing more context information.
Vulnerability Location	(6). Few-shot setting exposes LLM's limitations, and fine-tuning can greatly enhance the vulnerability location capabilities of LLMs. (7). Fine-tuning code-related LLMs as vulnerability locators is beneficial, as they can outperform pre-trained language models in terms of F1-score, precision, and FPR.
Vulnerability Description	(8). LLMs exhibit significantly weaker performance in generating vulnerability descriptions compared to pre-trained language models. Therefore, fine-tuning pre-trained language models for vulnerability detection is recommended.



(a) LLMs' performance on different software vulnerability tasks (∗ refers to the results under fine-tuning setting)

(b) The impacts of parameter sizes on LLMs' performance across different software vulnerability tasks

Fig. 2. The capability comparison of LLMs with different parameter sizes on different software vulnerability tasks.

LLMs are usually built on the transformer architecture [23] and can be classified into three types of architectures: encoder-only, encoder-decoder, and decoder-only. Encoder-only (e.g., CodeBERT [24], GraphCodeBERT [25], and UniXcoder [26]) and Encoder-Decoder (e.g., PLBART [27], CodeT5 [7], and CodeT5+ [8]) models are trained using Masked Language Modeling (MLM) or Masked Span Prediction (MSP) objective, respectively, where a small portion (e.g., 15%) of the tokens are replaced with either masked tokens or masked span tokens, LLMs are trained to recover the masked tokens. These models are trained as general ones on the code-related data and then are fine-tuned for the downstream tasks to achieve superior performance. Decoder-only models also attract a small portion of people's attention and they are trained by using Causal Language Modeling objectives to predict the probability of the next token given all previous tokens. GPT [2] and its variants are

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

the most representative models, which bring the large language models into practical usage.

Recently, the ChatGPT model attracts the widest attention from the world, which is the successor of the large language model InstructGPT [28] with a dialog interface that is fine-tuned using the Reinforcement Learning with Human Feedback (RLHF) approach [28], [29], [30]. RLHF initially fine-tunes the base model using a small dataset of prompts as input and the desired output, typically human-written, to refine its performance. Subsequently, a reward model is trained on a larger set of prompts by sampling outputs generated by the fine-tuned model. These outputs are then reordered by human labelers to provide feedback for training the reward model. Reinforcement learning [31] is then used to calculate rewards for each output generated based on the reward model, updating LLM parameters accordingly. With fine-tuning and alignment with human preferences, LLMs better understand input prompts and instructions, enhancing performance across various tasks [28], [32].

The application of LLMs in software engineering has seen a surge, with models like ChatGPT being employed for various tasks (e.g., code review, code generation, and vulnerability detection). Although some works use LLMs for vulnerability tasks [33], [34], our work differs from these previous studies in the following aspects. **(1) Closed-source ChatGPT vs. Open-source LLMs:** They only explore the capabilities of the closed-source ChatGPT in vulnerability tasks, whereas we investigate the abilities of both open-source code-related LLMs and general LLMs in these tasks. **(2) Prompts vs. Few-shot and Fine-tuning Settings:** They focus solely on the performance of LLMs using prompts, which introduces randomness and hinders the reproducibility of their findings. In contrast, we examine the capabilities of LLMs under both few-shot and fine-tuning settings, providing the source code and corresponding model files to ensure the reproducibility of our experimental results.

B. Software Vulnerability

Software Vulnerabilities (SVs) can expose software systems to risk situations and consequently make the software under cyber-attacks, eventually causing huge economic losses and even threatening people's lives. Therefore, vulnerability databases have been created to document and analyze publicly known security vulnerabilities. For example, Common Vulnerabilities and Exposures (CVE) [35], [36] and SecurityFocus [37] are two well-known vulnerability databases. Besides, Common Weakness Enumeration (CWE) defines the common software weaknesses of individual vulnerabilities, which are often referred to as vulnerability types of CVEs. To better address these vulnerabilities, researchers have proposed many approaches for understanding the effects of software vulnerabilities, including SV detection [38], [39], [40], [41], [42], [43], [44], [45], [46], [47], [48], [49], [50], SV assessment [20], [51], [52], [53], [54], SV location [55], [56], [57], SV repair [58], [59], [60], [61] as well as SV description [62], [63], [64], [65]. Many novel technologies are adopted to promote the progress of software vulnerability management, including software analysis [66], [67], machine learning [38], [45], and deep learning [51], [56], especially LLMs [63], [64].

III. Experimental Design

In this section, we present our studied dataset, our studied LLMs, the techniques for fine-tuning, the prompt engineering, the baseline approaches, the evaluation metrics, and the experiment settings.

A. Studied Dataset

We adopt the widely used dataset (named Big-Vul) provided by Fan et al. [21] by considering the following reasons. The most important one is to satisfy the distinct characteristics of the real world as well as the diversity in the dataset, which is suggested by previous works [45], [47]. Big-Vul, to the best of our knowledge, is the most large-scale vulnerability dataset with diverse information about the vulnerabilities, which are collected from practical projects and these vulnerabilities are recorded in the Common Vulnerabilities and Exposures (CVE)[1]. The second one is to compare fairly with existing state-of-the-art (SOTA) approaches (e.g., LineVul, Devign, and SVulD).

Big-Vul totally contains 3,754 code vulnerabilities collected from 348 open-source projects spanning 91 different vulnerability types from 2002 to 2019. It has 188,636 C/C++ functions with a vulnerable ratio of 5.7% (i.e., 10,900 vulnerability functions). The authors linked the code changes with CVEs and their descriptive information to enable a deeper analysis of the vulnerabilities.

We follow the same strategy to build the training data, validation data, and testing data from the original dataset with previous work does [39], [48], [58]. Specifically, 80% of functions are treated as training data, 10% of functions are treated as validation data, and the left 10% of functions are treated as testing data. We also keep the distribution as same as the original ones in training, validation, and testing data. Notice that we undersample the non-vulnerable functions to produce approximately balanced training data at the function level, while the validation and testing data remain in the original imbalanced ratio. To clean and normalize the dataset, we remove empty lines, leading and trailing spaces in each line, as well as comments from the source code. Finally, the split dataset is used for evaluation and the statistics are shown in Table II.

B. Studied LLMs

The general LLMs are pre-trained on textual data, including natural language and code, and can be used for a variety of tasks. In contrast, code-related LLMs are specifically pre-trained to automate code-related tasks. Due to the empirical nature of this work, we are interested in assessing the effectiveness of both LLM categories in vulnerability tasks. For the code-related LLMs, we select the top four models released

[1]https://cve.mitre.org/

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE II
STATISTIC OF THE STUDIED DATASET

Datasets	# Vul.	# Non-Vul.	# Total	% Vul.: Non-Vul.
Original Big-Vul	10,900	177,736	188,636	0.061
Filtered Big-Vul	5,260	96,308	101,568	0.055
Training	8,720	8,720	17,440	1
Validation	1,090	17,774	18,864	0.061
Testing	1,090	17,774	18,864	0.061

*We undersample the non-vulnerable functions to produce approximately balanced training data.

TABLE III
OVERVIEW OF THE STUDIED LLMS

Models	Code-related LLMs				General LLMs	
	DeepSeek-Coder	CodeLlama	StarCoder	WizardCoder	Mistral	Phi-2
Fine-Tuning	6.7B	7B	7B	7B	7B	2.7B
Few-Shot	6.7B & 33B	7B & 34B	7B & 34B	7B & 15.5B	7B	2.7B
Release Date	Nov'23	Aug'23	May'23	June'23	Sep'23	Dec'23

recently (in 2023), namely DeepSeek-Coder [9], CodeLlama [11], StarCoder [10], and WizardCoder [68]. For the general LLMs, we select the top two models, resulting in the selection of Mistral [69], and Phi-2 [70]. For the few-shot setting, we select the models with no more than 34B parameters from the Hugging Face Open LLM Leaderboard [71], as for the fine-tuning setting, we select the models with 7B parameters or less. The constraint on the number of parameters is imposed by our computing resources (i.e., 192GB RAM, 10 × NVIDIA RTX 3090 GPU). Table III summarizes the characteristics of the studied LLMs, we briefly introduce these LLMs to make our paper self-contained.

Group 1: Code-related LLMs. DeepSeek-Coder developed by DeepSeek AI [9] is composed of a series of code language models, each trained from scratch on 2T tokens, with a composition of 87% code and 13% natural language in both English and Chinese. They provide various sizes of the code model, ranging from 1B to 33B versions. Each model is pre-trained on project-level code corpus by employing a window size of 16K and an extra fill-in-the-blank task, to support project-level code completion and infilling. For coding capabilities, DeepSeek-Coder achieves state-of-the-art performance among open-source code models on multiple programming languages and various benchmarks.

CodeLlama proposed by Rozière et al. [11] is a set of large pre-trained language models for code built on Llama 2. They achieve state-of-the-art performance among open models on code tasks, provide infilling capabilities, support large input contexts, and demonstrate zero-shot instruction following for programming problems. CodeLlama is created by further training Llama 2 using increased sampling of code data. As with Llama 2, the authors applied extensive safety mitigations to the fine-tuned CodeLlama versions.

StarCoder proposed by Li et al. [10] is a large pre-trained language model specifically designed for code. It was pre-trained on a large amount of code data to acquire programming knowledge and trained on permissive data from GitHub, including over 80 programming languages, Git commits, GitHub issues, and Jupyter notebooks. StarCoder can perform code editing tasks, understand natural language prompts, and generate code that conforms to APIs. StarCoder represents the advancement of applying large language models in programming.

WizardCoder proposed by Luo et al. [68] is a large pre-trained language model that empowers Code LLMs with complex instruction fine-tuning, by adapting the Evol-Instruct method to the domain of code. Through comprehensive experiments on four prominent code generation benchmarks, namely HumanEval, HumanEval+, MBPP, and DS-1000, the authors unveil the exceptional capabilities of their model. It surpasses all other open-source Code LLMs by a substantial margin. Moreover, WizardCoder even outperforms the largest closed LLMs, Anthropic's Claude and Google's Bard, on HumanEval and HumanEval+.

Group 2: General LLMs. Mistral is a 7-billion-parameter language model released by Mistral AI [69]. Mistral 7B is a carefully designed language model that provides both efficiency and high performance to enable real-world applications. Due to its efficiency improvements, the model is suitable for real-time applications where quick responses are essential. At the time of its release, Mistral 7B outperformed the best open source 13B model (Llama 2) in all evaluated benchmarks.

Phi-2 proposed by Microsoft [70] packed with 2.7 billion parameters. It is designed to make machines think more like humans and do it safely. Phi-2 is not just about numbers; it is about a smarter, safer way for computers to understand and interact with the world. Phi-2 stands out because it is been taught with a mix of new language data and careful checks to make sure it acts right. It is built to do many things like writing, summarizing texts, and coding, but with better common sense and understanding than its earlier version, Phi-1.5. Phi-2's evaluation demonstrates its proficiency over larger models in aggregated benchmarks, emphasizing the potential of smaller models to achieve comparable or superior performance to their larger counterparts. This is particularly evident in its comparison with Google Gemini Nano 2, where Phi-2 outshines despite its smaller size.

C. Model Fine-Tuning

The four software vulnerability tasks can be categorized into two types: discriminative task (i.e., software vulnerability detection, software vulnerability assessment, and software vulnerability location) and generative task (i.e., software vulnerability description). Therefore, fine-tuning LLMs for software vulnerability tasks can be undertaken through both discriminative and generative methods, each method specifically designed to make LLMs aligned with the task. In particular, we treat the discriminative tasks as binary classification, while treating the generative task as generation one. The architectures for the two paradigms are presented in Fig. 3.

Discriminative Fine-Tuning. For vulnerability detection and vulnerability assessment, we utilize the *"AutoModelForSequenceClassification"* class provided by the Transformers library to implement discriminative fine-tuning. *"AutoModelForSequenceClassification"* is a generic model class that will

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

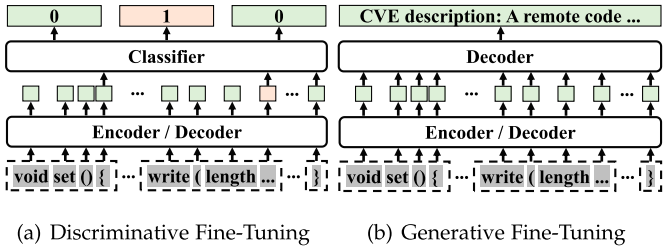


Fig. 3. Fine-tuning LLMs for software vulnerability tasks.

be instantiated as one of the sequence classification model classes of the library when created with the *"AutoModelForSequenceClassification.from_pretrained(model_name_or_path)"* class method.

For vulnerability location, we follow previous works [72], [73] that use LLMs to classify individual code lines as either vulnerable or non-vulnerable. For a token sequence $T = \{t_1, t_2, ..., t_n\}$ of the function, the model's decoder component, denoted as M, processes T to yield a sequence of output vectors: $O = M(T) = \{o_1, o_2, ..., o_L\}$, where O represents the output tensor with dimensions $L \times H$, L signifies the sequence length, and H denotes the hidden dimension size. During the process, the contextual information is captured by the masked self-attention mechanisms in the decoder of LLMs, where masked self-attention limits the sight to the preceding part of tokens. Each output vector o_i that represents the last token of one line is subsequently associated with a label (i.e., 0 or 1). The optimization process employs the binary cross-entropy as the loss function.

Generative Fine-Tuning. Generative fine-tuning aims to equip LLMs with the ability to perform Sequence-to-Sequence (Seq2Seq) tasks. Specifically, this involves inputting vulnerable code and generating the corresponding CVE descriptions related to the vulnerabilities. To calculate the loss during fine-tuning, we utilize the cross-entropy loss function, which is commonly used for Seq2Seq tasks. In this context, the loss measures the difference between the generated output sequence and the target sequence.

D. Prompt Engineering

For few-shot setting, we follow the prompt similar to those used in the artifacts, papers, or technical reports associated with each corresponding model [5], [10], [11], where each prompt contains three pieces of information: (1) task description, (2) source code, and (3) indicator. Using the software vulnerability detection task as an example, the prompt utilized for LLM consists of three crucial components, as depicted in Fig. 4:

- **Task Description** (marked as ①). We provide LLM with the description constructed as "`If this C code snippet has vulnerabilities, output Yes; otherwise, output No`". The task description used in the SV detection task varies based on the source programming language we employ.
- **Source Code** (marked as ②). We provide LLM with the code wrapped in "`// Code Start`" and "`// Code End`" Since we illustrate an example in C, we use the C comment format of "`//`" as a prefix for the description. We also employ different comment prefixes based on the programming language of the code.
- **Indicator** (marked as ③). We instruct LLM to think about the results. In this paper, we follow the best practice in previous work [12] and adopt the same prompt named "`// Detection`".

Prompt

1 Task Description:

If this C code snippet has vulnerabilities, output Yes; otherwise, output No.

2 Source Code:

```
// Code Start
void SendStatus(struct mg_connection* connection, const struct
mg_request_info* request_info, void* user_data) {
  std::string response = "HTTP/1.1 200 OK\r\n"
  "Content-Length:2\r\n\r\n"
  "ok";
  mg_write(connection, response.data(),
  response.length());
}
// Code End
```

3 Indicator:

```
// Detection
```

Fig. 4. The prompt contains three pieces of information: (1) task description, (2) source code, and (3) indicator.

Depending on the specific software vulnerability tasks, the task descriptions and indicators in the prompts may vary. The task descriptions and indicators for different software vulnerability tasks are presented in Table IV.

E. Baselines

To comprehensively compare the performance of LLMs with existing approaches, in this study, we consider the various pre-trained Language Models (LMs). As shown in Table V, these models have fewer than 220 million parameters and can be categorized into two categories: encoder-only LMs and encoder-decoder LMs. Encoder-only LMs (i.e., CodeBERT [24], GraphCodeBERT [25], and UniXcoder [26]) contain only the encoder component of a Transformer. They are designed for learning data representations and trained using the Masked Language Modeling (MLM) objective. Encoder-decoder LMs (i.e., PLBART [27], CodeT5 [7], and CodeT5+ [8]) have been proposed for sequence-to-sequence tasks. They are trained to recover the correct output sequence given the original input, often through span prediction tasks where random spans are replaced with artificial tokens. Recently, researchers have combined MLM with generative models for bidirectional and autoregressive text generation or infilling [74]. All these LMs can potentially be used for our tasks, so we evaluate these LMs.

For vulnerability location, we also consider Devign [38], Reveal [47], IVDetect [56], and LineVul [39] as baselines.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
THE TASK DESCRIPTIONS AND INDICATORS FOR DIFFERENT SOFTWARE VULNERABILITY TASKS

Dimension	Task Description	Indicator
Vulnerability Detection	If this C code snippet has vulnerabilities, output Yes; otherwise, output No.	// Detection
Vulnerability Assessment	Provide a qualitative severity ratings of CVSS v2.0 for the vulnerable C code snippet.	// Assessment
Vulnerability Location	Provide a vulnerability location result for the vulnerable C code snippet.	// Location
Vulnerability Description	Provide a CVE description for the vulnerable C code snippet.	// Description

TABLE V
OVERVIEW OF THE STUDIED LMS

Models	# Para.	Model Type	Models	# Para.	Model Type
CodeBERT	125M	Encoder-only LM	PLBART	140M	Encoder-decoder LM
GraphCodeBERT	125M	Encoder-only LM	CodeT5	220M	Encoder-decoder LM
UniXcoder	125M	Encoder-only LM	CodeT5+	220M	Encoder-decoder LM

*For UniXcoder, we use encoder-only mode.

In addressing vulnerability detection, we also include SVulD [48] in addition to the aforementioned approaches. We briefly introduce them as follows.

Devign proposed by Zhou et al. [38] is a general graph neural network-based model for graph-level classification through learning on a rich set of code semantic representations including AST, CFG, DFG, and code sequences. It uses a novel $Conv$ module to efficiently extract useful features in the learned rich node representations for graph-level classification.

Reveal proposed by Chakraborty et al. [47] contains two main phases. In the feature extraction phase, it translates code into a graph embedding, and in the training phase, it trains a representation learner on the extracted features to obtain a model that can distinguish the vulnerable functions from non-vulnerable ones.

IVDetect proposed by Li et al. [56] contains the coarse-grained vulnerability detection component and fine-grained interpretation component. In particular, IVDetect represents source code in the form of a program dependence graph (PDG) and treats the vulnerability detection problem as graph-based classification via graph convolution network with feature attention. As for interpretation, IVDetect adopts a GNNExplainer to provide fine-grained interpretations that include the subgraph in PDG with crucial statements that are relevant to the detected vulnerability.

LineVul proposed by Fu et al. [39] is a Transformer-based line-level vulnerability prediction approach. LineVul leverages BERT architecture with self-attention layers which can capture long-term dependencies within a long sequence. Besides, benefiting from the large-scale pre-trained model, LineVul can intrinsically capture more lexical and logical semantics for the given code input. Moreover, LineVul adopts the attention mechanism of BERT architecture to locate the vulnerable lines for finer-grained detection.

SVulD proposed by Ni et al. [48] is a function-level subtle semantic embedding for vulnerability detection along with heuristic explanations. Particularly, SVulD adopts contrastive learning to train the UniXcoder semantic embedding model for learning distinguishing semantic representation of functions regardless of their lexically similar information.

F. Evaluation Metrics

For considered software vulnerability-related tasks, we will perform evaluations using the widely adopted performance metrics. More precisely, to evaluate the effectiveness of LLMs on vulnerability detection and vulnerability assessment, we consider the following four metrics: F1-score, Recall, Precision, and Accuracy. Additionally, for vulnerability location, besides the four aforementioned metrics, we also consider the Top-k Accuracy and FPR metrics. For vulnerability description, we use *Rouge-1*, *Rouge-2*, and *Rouge-L* metrics.

G. Implementation

We develop the generation pipeline in Python, utilizing PyTorch [75] implementations of DeepSeek Coder, CodeLlama, StarCoder, WizardCoder, Mistral, and Phi-2. We use the Huggingface [76] to load the model weights and generate outputs. We also adhere to the best-practice guide [77] for each prompt. For the fine-tuning setting, we select the models with 7B parameters or less, and for the few-shot setting, we use models with fewer than 34B parameters. To directly compare the fine-tuning setting with the few-shot setting, we employ models with the same parameter in both settings (i.e., DeepSeek-Coder 6.7B, CodeLlama 7B, StarCoder 7B, WizardCoder 7B, Mistral 7B, and Phi-2 2.7B). The constraint on the number of parameters is imposed by our computing resources. Table III summarizes the characteristics of the studied LLMs. Furthermore, considering the limitation of LLM's conversation windows, we manually select three examples for the few-shot setting from the training data. Regarding baselines (i.e., pre-trained LMs, Reveal, IVDetect, Devign, LineVul, and SVulD), we utilize their publicly available source code and perform fine-tuning with the default parameters provided in their original code. Considering Devign's code is not publicly available, we make every effort to replicate its functionality and achieve similar results on the original paper's dataset. All these models are implemented using the PyTorch [75] framework. The evaluation is conducted on a 16-core workstation equipped with an Intel(R) Xeon(R) Gold 6226R CPU @ 2.90Ghz, 192GB RAM, and 10 × NVIDIA RTX 3090 GPU, running Ubuntu 20.04.1 LTS.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

IV. Experimental Results

This section presents the experimental results by evaluating LLMs performances on the widely used comprehensive dataset (i.e., Big-Vul [21]) covering four SV-related tasks.

A. RQ-1: Evaluating Vulnerability Detection of LLMs

In this RQ, we first investigate the vulnerability detection of LLMs and make a comparison with the existing state-of-the-art (SOTA) approaches. Then, we conduct a more detailed analysis of the results, comparing the detection performance of LLMs under the Top-10 CWE types.

Experimental Setting. We instruct LLMs with the following task description to tell it to act as a vulnerability detector.

> ***Task Description:*** *If this C code snippet has vulnerabilities, output Yes; otherwise, output No.*

In addition to pre-trained LMs, we also consider the following five SOTA baselines: Devign [38], Reveal [47], IVDetect [56], LineVul [39], and SVulD [48]. These baselines can be divided into two groups: graph-based (i.e., Devign, Reveal, and IVDetect) and transformer-based (i.e., pre-trained LMs, LineVul, and SVulD). Besides, in order to comprehensively compare the performance among baselines and LLMs, we consider four widely used performance measures (i.e., Precision, Recall, F1-score, and Accuracy) and conduct experiments on the popular dataset. Since graph-based approaches need to obtain the structure information (e.g., control flow graph (CFG), data flow graph (DFG)) of the studied functions, we adopt the same toolkit with *Joern* to transform functions. The functions are dropped out directly if they cannot be transformed by *Joern* successfully. Finally, the filtered dataset (shown in Table II) is used for evaluation. We follow the same strategy to build the training data, validation data, and testing data from the original dataset with previous work does [39], [58]. Specifically, 80% of functions are treated as training data, 10% of functions are treated as validation data, and the left 10% of functions are treated as testing data. We also keep the distribution as same as the original ones in training, validation, and testing data. We undersample the non-vulnerable functions to produce approximately balanced training data at the function level, while the validation and testing data remain in the original imbalanced ratio. Apart from presenting the overall performance comparison, we also give the detailed performance of LLMs on the Top-10 CWE types for a better analysis.

Results. [A] LLMs vs. Baselines. Table VI shows the overall performance measures between LLMs and eleven baselines and the best performances are highlighted in bold. According to the results in Table VI, we can obtain the following observations:

(1) Fine-tuned LLMs have poor performance compared with transformer-based approaches when considering *F1-score*, *Precision*, and *Accuracy*. In particular, SVulD obtains 0.336, 0.282, and 0.915 in terms of F1-score, Precision, and Accuracy, which surpass the fine-tuned LLMs by 24.4% to 57.0%, 64.0% to 108.9%, and 6.3% to 20.2% in terms of F1-score, Precision, and Accuracy, respectively. Notably, the F1-score performance of LineVul is significantly lower (0.272) than that reported in the original paper (0.910). We further analyze this discrepancy in Section V-A.

(2) The performance of fine-tuned LLMs is comparable to graph-based approaches. For example, in terms of F1-score, fine-tuned LLMs achieve a range of 0.214 to 0.270. In comparison, graph-based approaches achieve a range of 0.200 to 0.232.

(3) LLMs under few-shot setting have poor performance compared with baselines. LLMs ranging from 2.7B to 34B parameters perform less favorably than baselines in terms of F1-score and Precision. However, as for Accuracy, SVulD (transformer-based) obtains the best performance (0.915) and DeepSeek-Coder 6.7B under few-shot setting achieves a performance of 0.823, which is better than the three graph-based approaches.

> ***Finding-1***. *LLMs can detect vulnerabilities, but fine-tuned LLMs perform weaker than transformer-based approaches. Considering the computational resources and time costs of deploying LLMs, transformer-based approaches for vulnerability detection are a more efficient choice.*

[B] Fine-Tuning vs. Few-Shot. The experimental results are presented in Table VI. Based on these experimental findings,

TABLE VI
The Comparison Between LLMs and Eleven Baselines on Software Vulnerability Detection (RQ1)

Methods	F1-Score	Recall	Precision	Accuracy
Devign	0.200	0.660	0.118	0.726
Reveal	0.232	0.354	0.172	0.811
IVDetect	0.231	0.540	0.148	0.815
LineVul	0.272	0.620	0.174	0.828
SVulD	**0.336**	0.414	**0.282**	**0.915**
CodeBERT	0.270	0.608	0.173	0.830
GraphCodeBERT	0.246	0.721	0.148	0.771
UniXcoder	0.256	0.787	0.153	0.764
PLBART	0.255	0.692	0.157	0.791
CodeT5	0.237	0.759	0.141	0.748
CodeT5+	0.218	0.508	0.139	0.812
Fine-Tuning Setting				
DeepSeek-Coder 6.7B	0.270	0.627	0.172	0.824
CodeLlama 7B	0.259	**0.806**	0.154	0.761
StarCoder 7B	0.220	0.607	0.135	0.778
WizardCoder 7B	0.214	0.365	0.151	0.861
Mistral	0.220	0.607	0.135	0.778
Phi-2	0.241	0.557	0.154	0.818
Few-Shot Setting				
DeepSeek-Coder 6.7B	0.084	0.156	0.057	0.823
DeepSeek-Coder 33B	0.107	0.688	0.058	0.404
CodeLlama 7B	0.098	0.449	0.055	0.570
CodeLlama 34B	0.117	0.281	0.074	0.781
StarCoder 7B	0.094	0.443	0.053	0.560
StarCoder 15.5B	0.097	0.557	0.053	0.463
WizardCoder 7B	0.086	0.380	0.049	0.583
WizardCoder 34B	0.128	0.559	0.072	0.607
Mistral	0.126	0.401	0.074	0.711
Phi-2	0.099	0.563	0.054	0.471

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
THE SOFTWARE VULNERABILITY DETECTION COMPARISON ON TOP-10 CWES AMONG FINE-TUNED LLMS (RQ1)

CWE Type	# Total	# Vul.	DeepSeek-Coder	CodeLlama	StarCoder	WizardCoder	Mistral	Phi-2	DeepSeek-Coder	CodeLlama	StarCoder	WizardCoder	Mistral	Phi-2
			F1-Score						Precision					
CWE-119	1549	128	**0.321**	0.309	0.316	0.269	0.258	0.281	**0.223**	0.197	0.212	0.215	0.181	0.197
CWE-20	1082	80	0.269	**0.273**	0.229	0.216	0.145	0.269	0.173	0.163	0.141	0.165	0.096	**0.175**
CWE-264	800	64	**0.486**	0.468	0.337	0.348	0.356	0.477	0.357	0.316	0.232	0.308	0.269	**0.361**
CWE-399	697	35	**0.355**	0.286	0.209	0.274	0.227	0.306	**0.227**	0.169	0.125	0.191	0.143	0.196
CWE-125	582	29	0.233	**0.267**	0.213	0.195	0.179	0.180	0.145	**0.156**	0.129	0.128	0.108	0.109
CWE-200	573	27	**0.269**	0.261	0.241	0.180	0.162	0.229	**0.182**	0.159	0.151	0.132	0.106	0.152
CWE-189	442	21	0.235	0.208	0.255	0.273	0.178	**0.293**	0.145	0.119	0.151	0.180	0.108	**0.182**
CWE-362	413	16	0.031	**0.086**	0.075	0.050	0.026	0.032	0.017	**0.045**	0.040	0.029	0.014	0.018
CWE-416	406	12	**0.193**	0.178	0.148	0.145	0.146	0.141	**0.113**	0.101	0.083	0.093	0.086	0.082
CWE-476	367	11	0.091	**0.109**	0.053	0.057	0.037	0.019	0.049	**0.057**	0.028	0.032	0.020	0.010
CWE Type	# Total	# Vul.	DeepSeek-Coder	CodeLlama	StarCoder	WizardCoder	Mistral	Phi-2	DeepSeek-Coder	CodeLlama	StarCoder	WizardCoder	Mistral	Phi-2
			Recall						Accuracy					
CWE-119	1549	128	0.570	**0.719**	0.625	0.359	0.453	0.492	0.801	0.735	0.777	**0.839**	0.785	0.792
CWE-20	1082	80	0.609	**0.844**	0.609	0.313	0.297	0.578	0.804	0.735	0.758	**0.866**	0.793	0.814
CWE-264	800	64	0.763	**0.900**	0.613	0.400	0.525	0.700	0.839	0.795	0.759	**0.850**	0.810	0.846
CWE-399	697	35	0.815	**0.926**	0.630	0.481	0.556	0.704	0.885	0.821	0.815	**0.901**	0.854	0.877
CWE-125	582	29	0.586	**0.931**	0.621	0.414	0.517	0.517	0.808	0.746	0.771	**0.830**	0.763	0.765
CWE-200	573	27	0.514	**0.743**	0.600	0.286	0.343	0.457	0.829	0.743	0.770	**0.841**	0.784	0.812
CWE-189	442	21	0.625	**0.813**	**0.813**	0.563	0.500	0.750	0.853	0.776	0.828	**0.891**	0.833	0.869
CWE-362	413	16	0.200	**0.800**	0.600	0.200	0.200	0.200	0.847	0.794	0.821	**0.908**	0.821	0.855
CWE-416	406	12	0.667	**0.750**	0.667	0.333	0.500	0.500	0.835	0.796	0.773	**0.884**	0.828	0.820
CWE-476	367	11	0.571	**1.000**	0.429	0.286	0.286	0.143	0.782	0.687	0.706	**0.820**	0.719	0.725

we can draw the following observations: (1) LLMs fine-tuned for vulnerability detection demonstrate superior performance on the task compared to LLMs in the few-shot setting. The average F1-score and average Precision have doubled, while the average Recall has also shown improvement. (2) LLMs with more parameters typically exhibit better performance. For example, CodeLlama 34B improves upon CodeLlama 7B by 19.4%, 34.5%, and 37.0% in terms of F1-score, Precision, and Accuracy, respectively. However, different LLMs may exhibit performance variations due to differences in model design and the quality of pre-training data. (3) Phi-2 achieves performance approximating that of other LLMs with 7 billion parameters, even with a parameter size of 2.7 billion. This may be attributed to the higher quality of its pre-training data.

Finding-2. *After fine-tuning, the detection capability of LLMs has improved. Larger models usually perform better, but performance can also be influenced by model design and pre-training data. Therefore, fine-tuning the LLM on domain-specific data before using it as a vulnerability detector is necessary.*

[C] The comparisons of Top-10 CWE types between LLMs. Table VII shows the detailed comparisons of Top-10 CWE types between fine-tuned LLMs. In this table, we highlight the best performance for each performance metric in bold. According to the results, we can achieve the following observations: (1) In most cases, CodeLlama obtains better performance than other LLMs in terms of F1-score, Precision, and Recall. Different LLMs have certain advantages in different CWE types, complementing each other. (2) Considering the performance of F1-score, Precision, and Recall, CodeLlama achieves the best performances on CWE-125 (*"Out-of-bounds Read"*), CWE-362 (*"Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition')"*), and CWE-476 (*"NULL Pointer Dereference"*), which indicates CodeLlama is exceptionally skilled at detecting and mitigating vulnerabilities related to memory handling and synchronization issues.

Finding-3. *In general, different LLMs complementing each other, while CodeLlama obtains better performance in terms of F1-score, Precision, and Recall.*

B. RQ-2: Evaluating Vulnerability Assessment of LLMs

In this RQ, we delineate two task descriptions for vulnerability assessment: (1) code-based and (2) code-based with additional key information. We compare the performance of LLMs in both task descriptions for vulnerability assessment and concurrently conduct a case study to illustrate the effectiveness of incorporating key important information.

Experimental Setting. We instruct LLM with the following task descriptions (i.e., Task Description 1 and Task Description 2) to tell it to act as a vulnerability assessor. We first provide LLM with the vulnerable codes to explore its performance (Task Description 1). Moreover, we provide LLM with some key important information, including the CVE description, the project, the commit message as well as the file name when the vulnerable code exists to investigate the performance differences (Task Description 2).

Task Description 1: *Provide a qualitative severity rating of CVSS v2.0 for the vulnerable C code snippet.*
Task Description 2: *Provide a qualitative severity rating of CVSS v2.0 for the vulnerable C code snippet (with additional information).*

Results. Table VIII shows the detailed results of LLMs and six baselines on vulnerability assessment. Based on these experimental results, we can observe a significant improvement in the vulnerability assessment capability of LLMs after fine-tuning. Specifically, the accuracy has increased from 0.282 to 0.424, reaching a range of 0.759 to 0.860, while precision has improved from 0.296 to 0.355, now ranging from 0.512 to

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
THE COMPARISON BETWEEN LLMS AND SIX BASELINES ON SOFTWARE VULNERABILITY ASSESSMENT (RQ2)

Methods	F1-Score	Recall	Precision	Accuracy
CodeBERT	0.753	0.730	0.788	0.828
GraphCodeBERT	0.701	0.666	0.772	0.802
UniXcoder	0.745	0.761	0.734	0.817
PLBART	0.735	0.741	0.731	0.789
CodeT5	0.743	0.750	0.741	0.817
CodeT5+	0.706	0.677	0.755	0.789
Fine-Tuning Setting				
DeepSeek-Coder 6.7B	**0.814**	**0.785**	**0.854**	**0.860**
CodeLlama 7B	0.768	0.749	0.794	0.827
StarCoder 7B	0.671	0.677	0.666	0.764
WizardCoder 7B	0.793	0.778	0.813	0.842
Mistral	0.525	0.539	0.512	0.759
Phi-2	0.747	0.732	0.767	0.802
Few-Shot Setting				
DeepSeek-Coder 6.7B	0.229	0.339	0.310	0.262
DeepSeek-Coder 33B	0.290	0.323	0.336	0.335
CodeLlama 7B	0.310	0.331	0.334	0.373
CodeLlama 34B	0.265	0.323	0.327	0.294
StarCoder 7B	0.265	0.342	0.333	0.330
StarCoder 15.5B	0.285	0.315	0.329	0.326
WizardCoder 7B	0.244	0.351	0.336	0.250
WizardCoder 34B	0.306	0.330	0.325	0.379
Mistral	0.283	0.308	0.296	0.424
Phi-2	0.269	0.359	0.355	0.282

0.854. This underscores the necessity of fine-tuning in vulnerability assessment task. Overall, fine-tuned code-related LLMs outperform pre-trained LMs in vulnerability assessment. It is worth noting that DeepSeek-Coder, after fine-tuning, achieves the best performance compared to other LLMs and pre-trained LMs. If researchers need to perform tasks such as vulnerability assessment with LLM, fine-tuning DeepSeek-Coder is a more efficient choice. We also find that Mistral exhibits a relatively smaller improvement after fine-tuning, which aligns with our expectations, as it is a general LLM.

> ***Finding-4.*** *Overall, fine-tuned code-related LLMs outperform pre-trained LMs in vulnerability assessment. When resources permit, fine-tuning DeepSeek-Coder 6.7B for vulnerability assessment is optimal, as it outperforms the pre-trained LMs across four metrics.*

Case Study. To illustrate the effectiveness of key important information, we present an instance of a vulnerability (CWE-119) in Big-Vul that is exclusively assess by CodeLlama, as depicted in Table IX. This example is a vulnerability in the Linux project, categorized under CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer Vulnerability). In an initial assessment without critical information, CodeLlama did not fully grasp the severity of this vulnerability and labeled it as "Medium". However, with the provision of crucial details, CodeLlama can more accurately evaluate the risk level of this vulnerability. The CVE description for this vulnerability highlights multiple buffer overflows in the net/wireless/nl80211.c file of the Linux kernel prior to version 2.6.39.2. These vulnerabilities allow local users to gain elevated privileges by leveraging the CAP NET ADMIN capability during scan operations with an excessively long SSID value. In this scenario, the lack of proper validation of the SSID length leads to buffer overflows, enabling attackers to exploit the vulnerability, escalate privileges, and execute malicious code. The commit message described that this bug has existed since version 2.6.29-rc4 of the Linux kernel. Given this information, CodeLlama reassesses the risk level of this vulnerability as "High". This is because it allows attackers to escalate privileges and execute malicious code, and it has persisted for a considerable period of time. It is crucial to address and patch this vulnerability promptly by updating the operating system or kernel to ensure security.

To compare the vulnerability assessment capabilities of LLMs after providing key information, we have created a performance comparison bar chart, as shown in Fig. 5. LLMs have limited capacity for assessing vulnerability severity based solely on source code. However, when provided with key important information, most LLMs (i.e., DeepSeek-Coder, CodeLlama, WizardCoder, and Mistral) exhibit significantly improved vulnerability assessment capabilities, particularly in terms of the Accuracy metric. The Accuracy has increased from the range of 0.26-0.42 to the range of 0.27-0.56. StarCoder and Phi-2 are showing a declining trend, and we believe this may be attributed to the addition of key information, resulting in an increase in the number of input tokens. These LLMs may not excel in handling excessively long text sequences, and we analyze this further in Section V-B. In contrast, DeepSeek-Coder and Mistral exhibit significant improvements, possibly due to their proficiency in handling long sequential text.

> ***Finding-5.*** *LLMs have the capacity for assessment of vulnerability severity based on source code, and can be improved by providing more context information.*

C. RQ-3: Evaluating Vulnerability Location of LLMs

In this RQ, we first outline how to assess the vulnerability location capabilities of LLMs. Then, we proceed to compare the vulnerability location abilities of LLMs across different settings, both at a general level and in detail, and analyze the reasons behind the observed differences.

Experimental Setting. We select the vulnerable functions with information on vulnerable lines from the testing set for the evaluation and instruct LLM with the following task description to explore its vulnerability location performance.

> ***Task Description:*** *Provide a vulnerability location result for the vulnerable C code snippet.*

For the fine-tuning setting of LLMs and pre-trained LMs, we treat the vulnerability location task as a binary classification problem, determining whether each line of code is vulnerable or not. For the few-shot setting, a specific vulnerable function may contain one or several vulnerable lines, and LLM may also predict one or several potential vulnerable lines ($Lines_{predict}$).

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
A VULNERABLE CODE FOR CODELLAMA TO ASSESS WITH DIFFERENT PROMPTS (RQ2)

Improper Restriction of Operations within the Bounds of a Memory Buffer Vulnerability (CWE-119) in Linux	
Task Description 1	Provide a qualitative severity ratings of CVSS v2.0 for the vulnerable C code snippet.
Input 1	An example of a C code snippet with vulnerabilities (CVE-2011-2517).
Response 1	**Severity: Medium**
Task Description 2	Provide a qualitative severity rating of CVSS v2.0 for the vulnerable C code snippet (with additional information).
Input 2	**Project**: Linux **File Name**: net/wireless/nl80211.c **CVE Description**: Multiple buffer overflows in net/wireless/nl80211.c in the Linux kernel before 2.6.39.2 allow local users to gain privileges by leveraging the CAP_NET_ADMIN capability during scan operations with a long SSID value. **Commit Message**: nl80211: fix check for valid SSID size in scan operations. In both trigger_scan and sched_scan operations, we were checking for the SSID length before assigning the value correctly. Since the memory was just kzalloc'ed, the check was always failing and SSID with over 32 characters were allowed to go through. This was causing a buffer overflow when copying the actual SSID to the proper place. This bug has been there since 2.6.29-rc4.
Response 2	**Severity: High**
Analysis	The true Severity is High. After providing additional key information, CodeLlama output for the Severity changed from Medium to High.

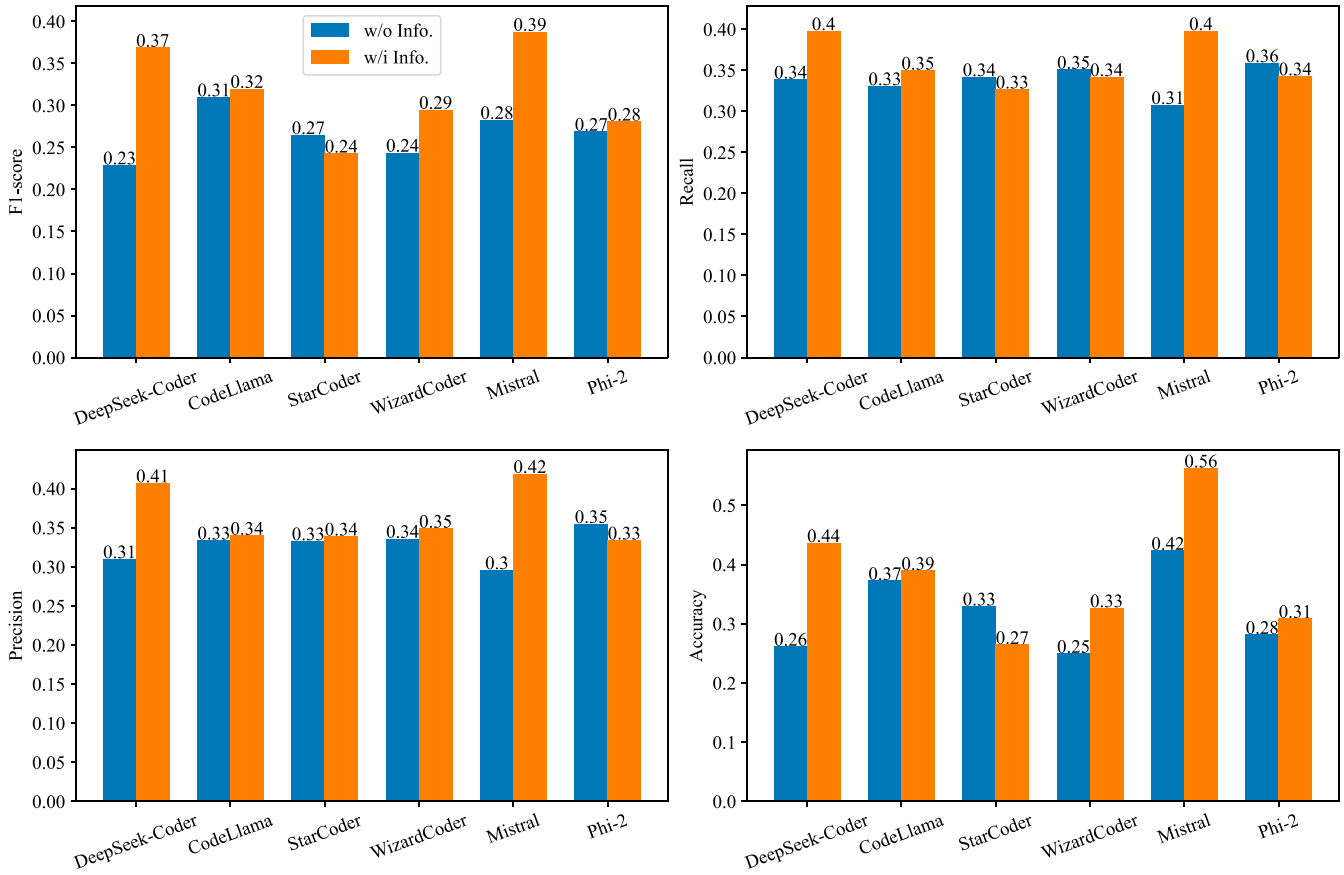


Fig. 5. The impact of key important information on LLM vulnerability assessment (RQ2).

We convert $Lines_{predict}$ into a binary classification format. For example, if a given vulnerable function consists of five lines and contains two vulnerable lines **[2, 3]**, and the LLM predicts one potential vulnerable line **[2]**, we convert this to a binary classification format as **[0, 0, 1, 0, 0]** for ease of computation. To better evaluate the vulnerability location performance of LLM on a specific vulnerable function, we consider five widely used performance measures (i.e., Precision, Recall, F1-score, Accuracy, and FPR).

In addition to pre-trained LMs, we also consider the following four SOTA baselines: Devign [38], Reveal [47], IVDetect [56], and LineVul [39]. For the graph-based approaches (i.e., Devign, Reveal, and IVDetect), we use GNNExplainer [78], [79] for vulnerability location. We compare the performance of

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE X
THE COMPARISON BETWEEN LLMS AND SIX BASELINES ON SOFTWARE VULNERABILITY LOCATION (RQ3)

Methods	F1-Score	Recall	Precision	Accuracy	FPR
CodeBERT	0.470	**0.514**	0.433	0.879	0.078
GraphCodeBERT	0.483	0.477	0.489	0.893	0.058
UniXcoder	0.460	0.384	0.575	0.908	0.032
PLBART	0.436	0.416	0.458	0.886	0.058
CodeT5	0.493	0.408	0.623	0.914	0.028
CodeT5+	0.303	0.207	0.565	0.902	**0.018**
Fine-Tuning Setting					
DeepSeek-Coder 6.7B	0.437	0.332	0.640	0.912	0.021
CodeLlama 7B	0.504	0.396	**0.691**	**0.919**	0.021
StarCoder 7B	0.245	0.169	0.443	0.893	0.024
WizardCoder 7B	**0.520**	0.427	0.664	0.918	0.025
Mistral	0.314	0.384	0.266	0.827	0.122
Phi-2	0.458	0.361	0.629	0.912	0.025
Few-Shot Setting					
DeepSeek-Coder 6.7B	0.111	0.111	0.112	0.852	0.081
DeepSeek-Coder 33B	0.110	0.112	0.108	0.849	0.084
CodeLlama 7B	0.082	0.063	0.116	0.882	0.043
CodeLlama 34B	0.115	0.090	0.158	0.884	0.044
StarCoder 7B	0.088	0.066	0.134	0.887	0.039
StarCoder 15.5B	0.095	0.078	0.120	0.876	0.052
WizardCoder 7B	0.082	0.063	0.120	0.884	0.042
WizardCoder 34B	0.096	0.072	0.145	0.887	0.039
Mistral	0.086	0.065	0.127	0.885	0.040
Phi-2	0.073	0.053	0.116	0.885	0.037

LLMs and these baselines using Top-k Accuracy, as employed in previous works [39], [79].

Results. Table X presents the overall performance of vulnerability location between LLMs and seven baselines. Based on this table, we can achieve the following observations: **(1) Fine-tuning can greatly enhance the vulnerability location capabilities of LLMs.** For example, after fine-tuning, CodeLlama 7B's F1-score increases from 0.082 to 0.504, recall increases from 0.063 to 0.396, precision increases from 0.116 to 0.691, accuracy increases from 0.882 to 0.919, and FPR decreases from 0.043 to 0.021. **(2) Code-related LLMs often outperform pre-trained LMs in terms of F1-score, precision, and FPR.** For example, CodeLlama 7B outperforms the pre-trained LMs, which are averaged over six different models, by 14.3%, 31.9%, and 54.3% in terms of F1-score, precision, and FPR, respectively.

The Top-k Accuracy results of the interpreters are shown in Fig. 6, where the x-axis represents k and the y-axis represents Top-k Accuracy (%). For comparison, we average the results of all LLMs and pre-trained LMs. We can observe that the performance of these graph-based methods does not show significant differences but is considerably weaker than that of pre-trained LMs, LLMs, and LineVul. Although pre-trained LMs achieve the highest accuracy at k=20, the difference between them and LLMs is not substantial.

Case Study. We find that LLMs under few-shot setting have limitations, and in some exceptional cases, they tend to output more vulnerable lines, even if these lines do not contain vulnerabilities. We take StarCoder as an example, Fig. 7 depicts a vulnerability code snippet from the Big-Vul dataset, with the vulnerability behavior occurring in lines 3 and 4. However, StarCoder tends to output more vulnerability lines, such as **"[1, 2, 3, 4, 5, 6, 7, 8, 9]"**, whereas after fine-tuning, StarCoder becomes more cautious and only predicts **"[4]"**. Note that we convert the model's predictions into a specific format, i.e., transform **"[0, 0, 0, 1, 0, 0, 0, 0, 0]"** to **"[4]"**.

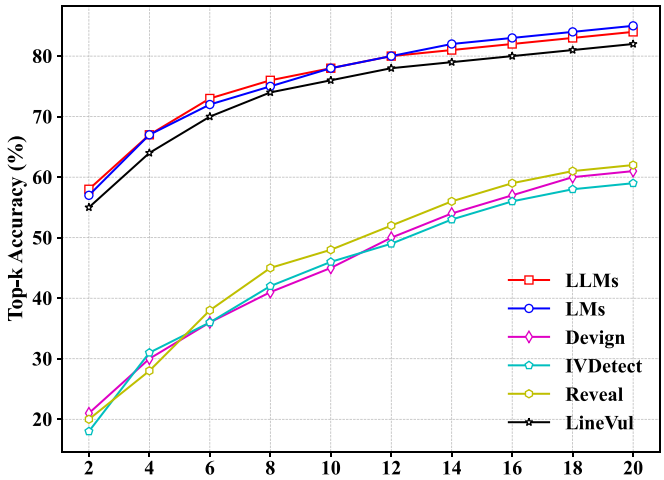


Fig. 6. The interpretation results for different vulnerability locators with k from 2 to 20 (RQ3).

Task Description:
Provide a vulnerability location result for the vulnerable C code snippet.

StarCoder: [1, 2, 3, 4, 5, 6, 7, 8, 9] StarCoder (Fine-Tuning): [4]

```
01 standard_info_part2(standard_display *dp,..., int nImages)
02 {
03     dp->pixel_size = bit_size(pp,..., png_get_bit_depth(pp, pi));
04     dp->bit_width = png_get_image_width(pp, pi) * dp->pixel_size;
05     dp->cbRow = png_get_rowbytes(pp, pi);
06     if (dp->cbRow != (dp->bit_width+7)/8)
07         png_error(pp, "bad png_get_rowbytes calculation");
08     store_ensure_image(dp->ps, pp, nImages, dp->cbRow, dp->h);
09 }
```

Fig. 7. An example to demonstrate the limitations of StarCoder in vulnerability location (RQ3).

> ***Finding-6.*** *Few-shot setting exposes LLM's limitations, and fine-tuning can greatly enhance the vulnerability location capabilities of LLMs.*
> ***Finding-7.*** *Fine-tuning code-related LLMs as vulnerability locators is beneficial, as they can outperform pre-trained LMs in terms of F1-score, precision, and FPR.*

D. RQ-4: Evaluating Vulnerability Description of LLMs

In this RQ, we employ the ROUGH metric to evaluate the LLMs' vulnerability description capabilities. We conduct a detailed statistical analysis of LLMs' abilities and also perform a case study to provide a comprehensive assessment of their performance in describing vulnerabilities.

Experimental Setting. We instruct LLMs with a designated task description, guiding them to perform the role of a vulnerability descriptor. Table XI illustrates an example of our approach to evaluating LLMs' proficiency in conducting vulnerability descriptions.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
AN EXAMPLE OF GENERATING VULNERABILITY DESCRIPTION FOR VULNERABLE CODE (RQ4)

Describe vulnerability code (CWE-119, Improper Restriction of Operations within the Bounds of a Memory Buffer)	
Task Description	Provide a CVE description for the vulnerable C code snippet.
Source Code	An example of a C code snippet with vulnerabilities. **CVE ID: CVE-2016-0841**.
Response From CodeT5+	libmedia **in mediaserver in Android 4.x before 4.4.4, 5.0.x before 5.0.2, 5.1.x before 5.1.1, and 6.x before** 2016-08-01 has certain incorrect declarations, **which allows remote attackers to execute arbitrary code or cause a denial of service** (NULL pointer dereference or **memory corruption) via a crafted media file, aka internal bug** 28166152.
Response From CodeLlama	**media/libmedia/MediaMetadataRetriever.cpp in mediaserver in Android 4.x before 4.4.4, 5.0.x before 5.0.2, 5.1.x before 5.1.1, and 6.x before 2016-04-01** does not validate the data size, **which allows attackers to execute arbitrary code or cause a denial of service (memory corruption) via a crafted media file, aka internal bug** 28799341. Specifically, the constructor calls the native function createMetadataRetriever in libstagefright, and later calls the getFrameAtTime function, aka internal bug 29244311. The getFrameAtTime call triggers a memory corruption warning, which allows an attacker to execute arbitrary code via a crafted video file, aka internal ...
CVE Description	media/libmedia/mediametadataretriever.cpp in mediaserver in Android 4.x before 4.4.4, 5.0.x before 5.0.2, 5.1.x before 5.1.1, and 6.x before 2016-04-01 mishandles cleared service binders, which allows remote attackers to execute arbitrary code or cause a denial of service (memory corruption) via a crafted media file, aka internal bug 26040840.

***Task Description**: Provide a CVE description for the vulnerable C code snippet.*

To evaluate the precision of the generated CVE description, we adopt the widely used performance metric ROUGE [80], which is a set of metrics and is used for evaluating automatic summarization and machine translation software in natural language processing. The metrics compare an automatically produced summary or translation against a reference or a set of references (human-produced) summary or translation. Here, we totally consider three settings: *1*, *2*, and *L*.

Results. Table XII represents the vulnerability description capabilities of LLMs and six baselines. According to the results, we can obtain the following observations: **(1) LLMs exhibit significantly weaker performance in generating vulnerability descriptions compared to pre-trained LMs.** For instance, pre-trained LMs achieve an average performance of 0.600, 0.487, and 0.591 on ROUGE-1, ROUGE-2, and ROUGE-L, respectively, whereas fine-tuned LLMs only achieve an average of 0.406, 0.301, and 0.400 on the same metrics. **(2) Fine-tuning can significantly enhance the performance of LLMs in vulnerability descriptions.** After fine-tuning, there is a several-fold improvement in ROUGE-1, ROUGE-2, and ROUGE-L. This suggests that these LLMs possess strong learning capabilities and can extract more gains from historical data. (3) The low ROUGE-2 scores indicate that Phi-2 has a limited ability to generate accurate and relevant high-order n-grams (pairs of consecutive words) in vulnerability descriptions, indicating potential issues in capturing specific and detailed information.

TABLE XII
THE COMPARISON OF LLMS ON SOFTWARE VULNERABILITY DESCRIPTION (RQ4)

Methods	**ROUGE-1**	**ROUGE-2**	**ROUGE-L**
CodeBERT	0.511	0.376	0.501
GraphCodeBERT	0.538	0.406	0.528
UniXcoder	0.658	0.558	0.650
PLBART	0.447	0.313	0.437
CodeT5	0.700	0.604	0.693
CodeT5+	**0.747**	**0.668**	**0.740**
Fine-Tuning Setting			
DeepSeek-Coder 6.7B	0.434	0.325	0.425
CodeLlama 7B	0.392	0.292	0.387
StarCoder 7B	0.420	0.321	0.416
WizardCoder 7B	0.425	0.327	0.419
Mistral	0.453	0.347	0.448
Phi-2	0.313	0.196	0.305
Few-Shot Setting			
DeepSeek-Coder 6.7B	0.230	0.073	0.215
DeepSeek-Coder 33B	0.219	0.066	0.203
CodeLlama 7B	0.221	0.070	0.205
CodeLlama 34B	0.258	0.094	0.242
StarCoder 7B	0.243	0.084	0.229
StarCoder 15.5B	0.255	0.089	0.241
WizardCoder 7B	0.230	0.066	0.211
WizardCoder 34B	0.276	0.111	0.261
Mistral	0.290	0.095	0.267
Phi-2	0.210	0.056	0.194

Case Study. To demonstrate the capability of pre-trained LMs and LLMs in generating vulnerability descriptions, we present an example of a vulnerability (CWE-119) described by CodeT5+ and CodeLlama, as shown in Table XI. This example represents a vulnerability within the Linux project, categorized as CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer Vulnerability). It is noteworthy that even when provided with only the code of the vulnerability, CodeT5+ produces text highly similar to the CVE description (highlighted in orange), indicating that pre-trained LMs are capable of comprehending the essence and crucial features of vulnerabilities and expressing this information in natural language.

Additionally, we find that CodeLlama's response is very similar to the CVE description, but with many additional details. We hypothesize that the poor performance of LLMs is not due to

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
THE COMPARISON OF CODET5+, CODELLAMA, AND CODELLAMA-TRIM ON SELECTED EXAMPLES (RQ4)

Methods	ROUGE-1	ROUGE-2	ROUGE-L
CodeT5+	0.730	0.644	0.722
CodeLlama	0.366	0.266	0.360
CodeLlama-Trim	0.625	0.523	0.616

TABLE XIV
THE REPRODUCED RESULTS FOR LINEVUL

Datasets	F1-Score	Accuracy	Recall	Precision
Original dataset	0.90	0.95	0.86	0.95
Remove empty lines	0.85	0.98	0.79	0.93
Remove comments	0.86	0.99	0.81	0.93
Remove spaces	**0.40**	**0.94**	**0.37**	**0.45**

their inability to generate appropriate vulnerability descriptions, but rather because they tend to output tokens endlessly, even when they should stop. In contrast, pre-trained LMs typically stop at the appropriate points.

To further analyze this, we investigate the vulnerability description capabilities of LLMs after mitigating this issue. Using CodeLlama as an example, we randomly select 100 examples from the testing set and manually determine where the descriptions should terminate, trimming CodeLlama's output accordingly. We then calculate the ROUGE metrics for the trimmed outputs and compare them with the original results and those of CodeT5+. The final results are presented in Table XIII, we find that after trimming, the ROUGE-1, ROUGE-2, and ROUGE-L scores for CodeLlama significantly improved, even nearing those of CodeT5+. This confirms our hypothesis that LLMs actually possess strong vulnerability description capabilities, but their performance is hindered by the tendency to output excessively.

Finding-8. *LLMs exhibit significantly weaker performance in generating vulnerability descriptions compared to pre-trained LMs. Therefore, fine-tuning pre-trained LMs for vulnerability detection is recommended.*

V. DISCUSSION

This section discusses open questions regarding the performance differences observed, the impact of input sequence length, and potential threats to the validity of our results.

A. Analysis of Performance Difference

In RQ1, for LineVul, there is a huge difference between the results obtained in this paper (i.e., 0.272 of F1-score) and the ones reported in original work (i.e., 0.910 of F1-score). We first check the correctness of our LineVul reproduction by re-conducting the corresponding experiments using the original dataset provided by LineVul's official source and we obtain similar results. Then, we inspect each step of the data pre-processing, as outlined in Section III-A. In particular, this process involves three pre-processing in total: removing blank lines, removing comments, and trimming leading and trailing spaces from lines. We pre-process the original dataset of LineVul, re-train, and test the model under the same parameter settings. The results are shown in Table XIV and we obtain the following conclusions:

- Our reproduced LineVul performs closely to the original one.
- Removing blank lines and comments does not significantly affect LineVul's results.
- Trimming leading and trailing spaces from lines causes a drastic decrease in LineVul's performance.

TABLE XV
THE COMPARISON BETWEEN LLMS AND ELEVEN BASELINES ON THE FILTERED BIG-VUL WITHOUT PRE-PROCESSING

Methods	F1-Score	Recall	Precision	Accuracy
Devign	0.205	0.672	0.121	0.730
Reveal	0.237	0.367	0.175	0.878
IVDetect	0.233	0.547	0.148	0.813
LineVul	0.743	0.789	0.702	**0.972**
SVulD	**0.744**	0.774	**0.716**	**0.972**
CodeBERT	0.590	0.837	0.460	0.940
GraphCodeBERT	0.703	0.757	0.656	0.967
UniXcoder	0.670	0.869	0.545	0.956
PLBART	0.258	0.504	0.173	0.850
CodeT5	0.644	0.878	0.508	0.950
CodeT5+	0.691	0.878	0.569	0.959
Fine-Tuning Setting				
DeepSeek-Coder 6.7B	0.651	0.913	0.506	0.949
CodeLlama 7B	0.554	**0.924**	0.395	0.923
StarCoder 7B	0.571	0.873	0.424	0.932
WizardCoder 7B	0.583	0.894	0.432	0.934
Mistral	0.448	0.591	0.361	0.925
Phi-2	0.697	0.859	0.586	0.961

In this section, we aim to evaluate the impact of data pre-processing on the fine-tuning of both the LLMs and the baselines. In RQ1, we have compared LLMs against baselines (i.e., graph-based approaches and transformer-based approaches) on the filtered Big-Vul with pre-processing (refer to Section III-A for more details). Therefore, we conduct additional experiments on (1) the filtered Big-Vul without pre-processing, (2) the entire Big-Vul without pre-processing, and (3) the entire Big-Vul with pre-processing.

From Tables XV, XVI, and XVII, we can draw conclusions similar to those for RQ1: (1) Fine-tuned LLMs perform poorly compared to transformer-based approaches. (2) Among the transformer-based approaches, LineVul and SVulD demonstrate strong performance across F1-score, Precision, and Accuracy. Notably, the PLBartTokenizer removes some unnecessary spaces, so whether the data is pre-processed does not have a significant impact on the PLBART model.

By comparing between Tables XV and VI as well as between Tables XVI and XVII, we observe a noticeable performance difference in transformer-based approaches and fine-tuned LLMs depending on whether the Big-Vul dataset is pre-processed. This difference may stem from two factors:

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

TABLE XVI
THE COMPARISON BETWEEN LLMS AND EIGHT BASELINES ON THE ENTIRE BIG-VUL WITHOUT PRE-PROCESSING

Methods	F1-Score	Recall	Precision	Accuracy
LineVul	0.753	0.801	0.710	0.970
SVulD	**0.757**	0.793	**0.724**	**0.971**
CodeBERT	0.623	0.895	0.478	0.938
GraphCodeBERT	0.741	0.765	0.718	0.969
UniXcoder	0.714	**0.921**	0.583	0.957
PLBART	0.265	0.695	0.164	0.777
CodeT5	0.739	0.824	0.669	0.966
CodeT5+	0.720	0.904	0.599	0.959
Fine-Tuning Setting				
DeepSeek-Coder 6.7B	0.739	0.886	0.633	0.964
CodeLlama 7B	0.702	0.850	0.598	0.958
StarCoder 7B	0.666	0.784	0.579	0.955
WizardCoder 7B	0.695	0.882	0.574	0.955
Mistral	0.553	0.873	0.405	0.919
Phi-2	0.696	0.896	0.569	0.955

TABLE XVII
THE COMPARISON BETWEEN LLMS AND EIGHT BASELINES ON THE ENTIRE BIG-VUL WITH PRE-PROCESSING

Methods	F1-Score	Recall	Precision	Accuracy
LineVul	0.307	0.583	0.209	0.848
SVulD	**0.332**	0.439	**0.267**	**0.898**
CodeBERT	0.289	0.630	0.187	0.821
GraphCodeBERT	0.299	0.649	0.194	0.824
UniXcoder	0.302	0.590	0.203	0.842
PLBART	0.255	0.687	0.157	0.769
CodeT5	0.285	0.733	0.177	0.788
CodeT5+	0.275	0.617	0.177	0.812
Fine-Tuning Setting				
DeepSeek-Coder 6.7B	0.268	**0.805**	0.161	0.747
CodeLlama 7B	0.289	0.773	0.178	0.781
StarCoder 7B	0.251	0.662	0.155	0.772
WizardCoder 7B	0.291	0.756	0.180	0.787
Mistral	0.271	0.364	0.216	0.887
Phi-2	0.281	0.675	0.177	0.800

(1) The pre-processed code may have a format that differs from the pre-training data, potentially impacting performance. (2) The code format helps LLMs and transformer-based approaches better understand the semantics of the programs, as it indicates the structure and relationships between statements.

B. Analysis of Input Sequence Length

In RQ2, we find that after adding key information, the performance of StarCoder and Phi-2 in vulnerability assessment actually weakened. We hypothesize that these LLMs may not excel in handling excessively long text sequences. Therefore, adding key information, which results in an increase in the number of input tokens, leads to a decline in performance. In this section, we aim to analyze the performance of StarCoder and Phi-2 with respect to input sequence length to determine whether there is a performance decline as the input length increases. As shown in Fig. 8, the horizontal axis represents the token length of the input sequence, and the vertical axis represents the F1-score of vulnerability assessment. We categorize the input token lengths into 0-128, 128-256, 256-512, 512-1024, and 1024+ (e.g., an input token length of 64 falls into the 0-128 range), and evaluate the vulnerability assessment performance of StarCoder and Phi-2 for each category. According to Fig. 8, we observe that as the input length increases, the F1-scores of both LLMs gradually decrease, revealing their significant limitations in assessing long sequences of vulnerable code. Therefore, in practical applications requiring the assessment of long sequences of vulnerable code, we may need to consider alternative optimization strategies or model choices to ensure accuracy and reliability.

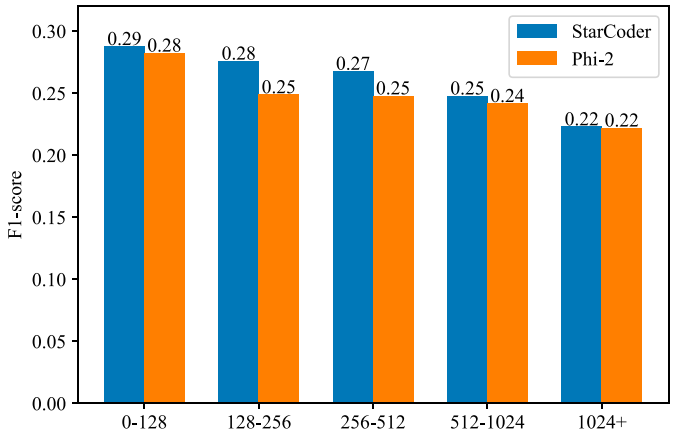


Fig. 8. The variation of F1-scores for vulnerability assessment with respect to input sequence length.

C. Threats to Validity

Threats to Internal Validity mainly contains in two-folds. The first one is the design of a prompt to instruct LLMs to give out responses. We design our prompt according to the practical advice [77] which has been verified by many users online and can obtain a good response from LLMs. Furthermore, LLMs will generate responses with some randomness even given the same prompt. Therefore, we set "temperature" to 0, which will reduce the randomness at most and we try our best to collect all these results in two days to avoid the model being upgraded. The second one is about the potential mistakes in the implementation of studied baselines. To minimize such threats, we directly use the original source code shared by corresponding authors.

Threats to External Validity may correspond to the generalization of the studied dataset. To mitigate this threat, we adopt the most large-scale vulnerability dataset with diverse information about the vulnerabilities, which are collected from practical projects, and these vulnerabilities are recorded in the Common Vulnerabilities and Exposures (CVE). However, we do not consider these vulnerabilities found recently. Besides, we do not adopt another large-scale vulnerability dataset named SARD since it is built manually and cannot satisfy the distinct characteristics of the real world [45], [47].

Threats to Construct Validity mainly correspond to the performance metrics in our evaluations. To minimize such threats, we consider a few widely used performance metrics to evaluate

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

the performance of LLMs on different types of tasks, e.g., Recall, Precision, and ROUGE.

VI. Conclusion

This paper aims to comprehensively investigate the capabilities of LLMs for software vulnerability tasks as well as its impacts. To achieve that, we adopt a large-scale vulnerability dataset (named Big-Vul) and then conduct several experiments focusing on four dimensions: **(1) Vulnerability Detection**, **(2) Vulnerability Assessment**, **(3) Vulnerability Location**, and **(4) Vulnerability Description**. Overall, although LLMs show some ability in certain areas, they still need further improvement to be competent in software vulnerability-related tasks. Our research conducts a comprehensive survey of LLMs' capabilities and provides a reference for enhancing its understanding of software vulnerabilities in the future.

References

[1] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[2] A. Radford et al., "Improving language understanding by generative pre-training," 2018.

[3] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surveys*, vol. 55, no. 9, pp. 1–35, 2023.

[4] R. Tang, Y.-N. Chuang, and X. Hu, "The science of detecting LLM-generated texts," 2023, *arXiv:2303.07205*.

[5] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," 2022, *arXiv:2203.13474*.

[6] Q. Zheng et al., "CodeGeeX: A pre-trained model for code generation with multilingual evaluations on HumanEval-X," 2023, *arXiv:2303.17568*.

[7] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.

[8] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, "CodeT5+: Open code large language models for code understanding and generation," 2023, *arXiv:2305.07922*.

[9] "DeepSeek coder: Let the code write itself," GitHub. Accessed: Nov. 3, 2023. [Online]. Available: https://github.com/deepseek-ai/DeepSeek-Coder

[10] R. Li et al., "StarCoder: May the source be with you!" 2023, *arXiv:2305.06161*.

[11] B. Rozière et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.

[12] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA:: ACM, 2023.

[13] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT," 2023, *arXiv:2304.00385*.

[14] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 959–971.

[15] R. Pan et al., "Understanding the effectiveness of large language models in code translation," 2023, *arXiv:2308.03109*.

[16] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring llm-based general bug reproduction," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 2312–2323.

[17] D. Zan et al., "Large language models meet NL2code: A survey," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics* (Long Papers), vol. 1, 2023, pp. 7443–7464.

[18] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proc. Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 919–931.

[19] S. Khan and S. Parkinson, "Review into state of the art of vulnerability assessment using artificial intelligence," in *Guide to Vulnerability Analysis for Computer Networks and Systems*, Springer, 2018, pp. 3–32.

[20] T. H. Le, H. Chen, and M. A. Babar, "A survey on data-driven software vulnerability assessment and prioritization," *ACM Comput. Surv.*, vol. 55, no. 5, pp. 1–39, 2022.

[21] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proc. 17th Int. Conf. Mining Softw. Repositories*, 2020, pp. 508–512.

[22] "Replication," GitHub, 2024. Accessed: Jan. 22, 2024. [Online]. Available: https://github.com/vinci-grape/VulEmpirical

[23] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.

[24] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

[25] D. Guo et al., "GraphCodeBERT: Pre-training code representations with data flow," 2020, *arXiv:2009.08366*.

[26] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," 2022, *arXiv:2203.03850*.

[27] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," 2021, *arXiv:2103.06333*.

[28] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 27730–27744.

[29] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.

[30] D. M. Ziegler et al., "Fine-tuning language models from human preferences," 2019, *arXiv:1909.08593*.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[32] Y. Bang et al., "A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity," 2023, *arXiv:2302.04023*.

[33] X. Zhou, T. Zhang, and D. Lo, "Large language model for vulnerability detection: Emerging results and future directions," in *Proc. ACM/IEEE 44th Int. Conf. Softw. Eng.: New Ideas Emerg. Results*, 2024, pp. 47–51.

[34] M. Fu, C. K. Tantithamthavorn, V. Nguyen, and T. Le, "ChatGPT for vulnerability detection, classification, and repair: How far are we?" in *Proc. 30th Asia-Pacific Softw. Eng. Conf. (APSEC)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 632–636.

[35] "Common vulnerabilities and exposures (CVE)," CVE MITRE, 2023. Accessed: Dec. 8, 2023. [Online]. Available: https://cve.mitre.org/

[36] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software," in *Proc. 17th Int. Conf. Predictive Models Data Analytics Softw. Eng.*, 2021, pp. 30–39.

[37] "SecurityFocus," Symantec, 2023. [Online]. Available: https://www.securityfocus.com/

[38] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 10197–10207.

[39] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction,"in *Proc. 19th Int. Conf. Mining Softw. Repos.*, 2022, pp. 608–620.

[40] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks," 2022, *arXiv:2203.02660*.

[41] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A framework for using deep learning to detect software vulnerabilities," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2244–2258, Jul./Aug. 2022.

[42] X. Cheng, G. Zhang, H. Wang, and Y. Sui, "Path-sensitive code embedding via contrastive learning for software vulnerability detection," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 519–531.

[43] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "VulCNN: An image-inspired scalable vulnerability detection system," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 2365–2376.

[44] Z. Li et al., "VulDeePecker: A deep learning-based system for vulnerability detection," in *Proc. 25th Annu. Netw. Distrib. System Secur. Symp.*, 2018, pp. 1–15.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.

[45] D. Hin, A. Kan, H. Chen, and M. A. Babar, "LineVD: Statement-level vulnerability detection using graph neural networks," 2022, *arXiv:2203.05181*.

[46] X. Zhan et al., "ATVHunter: Reliable version detection of third-party libraries for vulnerability identification in android applications," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1695–1707.

[47] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.

[48] C. Ni, X. Yin, K. Yang, D. Zhao, Z. Xing, and X. Xia, "Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2023, pp. 1611–1622.

[49] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 2237–2248.

[50] B. Steenhoek, H. Gao, and W. Le, "Dataflow analysis-inspired deep learning for efficient vulnerability detection," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[51] C. Ni, L. Shen, W. Wang, X. Chen, X. Yin, and L. Zhang, "FVA: Assessing function-level vulnerability by integrating flow-sensitive structure and code statement semantic," in *Proc. IEEE/ACM 31st Int. Conf. Program Comprehension (ICPC)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 339–350.

[52] A. Feutrill, D. Ranathunga, Y. Yarom, and M. Roughan, "The effect of common vulnerability scoring system metrics on vulnerability exploit delay," in *Proc. 6th Int. Symp. Comput. Netw. (CANDAR)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 1–10.

[53] G. Spanos and L. Angelis, "A multi-target approach to estimate software vulnerability characteristics and severity scores," *J. Syst. Softw.*, vol. 146, pp. 152–166, Dec. 2018.

[54] T. H. M. Le, D. Hin, R. Croft, and M. A. Babar, "DeepCVA: Automated commit-level vulnerability assessment with deep multi-task learning," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 717–729.

[55] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "VulDeeLocator: A deep learning-based fine-grained vulnerability detector," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2821–2837, Jul./Aug. 2022.

[56] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 292–303.

[57] C. Ni, W. Wang, K. Yang, X. Xia, K. Liu, and D. Lo, " The best of both worlds: Integrating semantic features with expert features for defect prediction and localization," in *Proc. 30th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, New York, NY, USA: ACM, 2022, pp. 672–683.

[58] C. Ni, K. Yang, X. Xia, D. Lo, X. Chen, and X. Yang, "Defect identification, categorization, and repair: Better together," 2022, *arXiv:2204.04856*.

[59] Q. Zhang, Y. Zhao, W. Sun, C. Fang, Z. Wang, and L. Zhang, "Program repair: Automated vs. manual," 2022, *arXiv:2203.05166*.

[60] Z. Chen, S. J. Kommrusch, M. Tufano, L.-N. Pouchet, D. Poshyvanyk, and M. Monperrus, "Sequencer: Sequence-to-sequence learning for end-to-end program repair," *IEEE Trans. Softw. Eng.*, vol. 47, no. 9, pp. 1943–1959, Sep. 2021.

[61] Q. Zhu et al., "A syntax-guided edit decoder for neural program repair," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 341–353.

[62] J. Sun et al., "Generating informative CVE description from exploitdb posts by extractive summarization," *ACM Trans. Softw. Eng. Methodol.*, 2021, *arXiv:2101.01431*.

[63] H. Guo, S. Chen, Z. Xing, X. Li, Y. Bai, and J. Sun, "Detecting and augmenting missing key aspects in vulnerability descriptions," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, pp. 1–27, 2022.

[64] H. Guo, Z. Xing, S. Chen, X. Li, Y. Bai, and H. Zhang, "Key aspects augmentation of vulnerability description based on multiple security databases," in *Proc. IEEE 45th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1020–1025.

[65] H. Guo, Z. Xing, and X. Li, "Predicting missing information of key aspects in vulnerability reports," 2020, *arXiv:2008.02456*.

[66] G. Fan, R. Wu, Q. Shi, X. Xiao, J. Zhou, and C. Zhang, "Smoke: Scalable path-sensitive memory leak detection for millions of lines of code," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 72–82.

[67] W. Li, H. Cai, Y. Sui, and D. Manz, "PCA: Memory leak detection using partial call-path analysis," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 1621–1625.

[68] Z. Luo et al., "WizardCoder: Empowering code large language models with evol-instruct," 2023, *arXiv:2306.08568*.

[69] A. Q. Jiang et al., "Mistral 7b," 2023, *arXiv:2310.06825*.

[70] "Phi-2: The surprising power of small language models," Microsoft, 2023. Accessed: Dec. 4, 2023. [Online]. Available: https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/

[71] "Hugging face open LLM leaderboard," Hugging Face, 2023. [Online]. Available: https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard

[72] A. Z. Yang, C. Le Goues, R. Martins, and V. Hellendoorn, "Large language models for test-free fault localization," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–12.

[73] J. Zhang et al., "An empirical study of automated vulnerability localization with large language models," 2024, *arXiv:2404.00287*.

[74] A. Aghajanyan et al., "CM3: A causal masked multimodal model of the internet," 2022, *arXiv:2201.07520*.

[75] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

[76] "Hugging face," Hugging Face, 2023. Accessed: Nov. 3, 2023. [Online]. Available: https://huggingface.co

[77] J. Shieh, "Best practices for prompt engineering with OpenAI API," *OpenAI*. [Online]. Available: https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-openai-api

[78] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 9244–9255.

[79] Y. Hu et al., "Interpreters for GNN-based vulnerability detection: Are we there yet?" in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2023, pp. 1407–1419.

[80] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, 2004, pp. 74–81.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on July 18,2025 at 09:10:28 UTC from IEEE Xplore. Restrictions apply.